-
-
-
-

DOCUMENT DE RÉFÉRENCE — ARCHITECTURE & TRAVAIL EN ÉQUIPE (PROJET DJANGUI)

1. Vue d'ensemble et Stack Technique

Djangui est une application web SaaS de gestion de tontines communautaires au Cameroun, conçue pour fonctionner sur **Smartphone (Mobile-First)** et sur **Ordinateur (Desktop)**.

- **Backend :** PHP 8.2+ en architecture **procédurale simple** (sans POO complexe ni framework), utilisant uniquement l'objet `$pdo` avec des requêtes préparées.
- **Base de données :** MySQL / MariaDB (`djangui_db` sous XAMPP) structurée en **10 tables** normalisées.
- **Frontend :** HTML5, **Bootstrap 5.3.3**, **Bootstrap Icons 1.11.3**, police **Inter** et feuille de style personnalisée `assets/css/style.css`.
- **Système Bi-Thème (Mode Clair & Mode Sombre) :**
  - Le fichier `includes/header.php` lit `localStorage.getItem('djangui_theme')` (par défaut `'light'`) et applique les attributs `data-theme` et `data-bs-theme` sur la balise `<html>` avant le rendu de la page.

- Le Mode Clair offre une haute lisibilité (idéal pour les aînés et l'usage en plein jour), tandis que le Mode Sombre applique une ambiance Bleu Nuit profond (`#070b19`) et Gris Ardoise (`#111827`).

- **Navigation Responsive Mobile & PC :**
  - Sur **PC (`lg` ≥ 992px)** : barre de navigation supérieure complète (`.navbar-djangui`).
  - Sur **Smartphone (`< 992px`)** : le bouton hamburger est désactivé (`d-none`) et remplacé par une barre d'application fixe en bas de l'écran (`.mobile-bottom-nav d-lg-none`).

2. Arborescence Officielle du Projet (`htdocs/djangui/`)

Chaque fichier a un emplacement fixe. **Aucun membre ne doit renommer un dossier ou déplacer un fichier commun.**

```
djangui/
├── ARCHITECTURE_ET_EQUIPE.md         # Ce document de référence pour les 5 membres
├── .gitignore                        # Ignore les fichiers locaux inutiles
│
├── database/
│   └── djangui_complet.sql           # Script SQL des 10 tables + données de démo
│
├── assets/
│   ├── css/
│   │   └── style.css                 # Design System Bi-Thème (Clair / Sombre) verrouillé
│   ├── js/
│   │   └── script.js                 # Bascule Clair/Sombre + Calculs dynamiques JS
│   └── video/
│       └── djangui.mp4               # Vidéo locale optionnelle de la page d'accueil
│
├── includes/                         # SOCLE COMMUN (Inclus dans toutes les pages)
│   ├── db.php                        # Connexion PDO MySQL ($pdo)
│   ├── functions.php                 # Fonctions globales (e, format_fcfa, verifier_connexion, CSRF...)
│   ├── header.php                    # <head>, détection du thème Clair/Sombre, CDN Bootstrap 5.3
│   ├── navbar.php                    # Barre du haut + Barre mobile du bas + Alertes Flash
│   ├── footer.php                    # Pied de page (.footer-djangui) + Chargement des scripts JS
│   └── waiting.php              # Page d'attente d'approbation par l'administrateur
│
├── admin/                            # ESPACE SECRÉTAIRE / ADMINISTRATEUR (12 pages)
│   ├── dashboard.php                 # Vue globale, KPIs, bouffeur du tour, solde caisse
│   ├── tontine-info.php              # Infos générales, code d'invitation, clôture définitive
│   ├── members.php       # Validation des adhésions (en_attente -> accepte/refuse) et gestion des membres (CNI visible ici)
│   ├── cycles.php                    # Création, configuration, activation et clôture de cycles
│   ├── draws.php                     # Tirage au sort aléatoire verrouillé et suivi des pots
│   ├── contributions.php             # Suivi des cotisations par séance et reliquats
│   ├── penalties.php                 # Gestion de la Caisse de réserve (pénalités & bonus)
│   ├── meetings.php                  # Programmation et publication des réunions Zoom
│   ├── reports.php                   # Bilans financiers et historique multi-cycles
│   ├── subscription.php              # Abonnement SaaS (2 mois gratuits puis 2 000 FCFA/mois)
│   ├── transfer.php                  # Passation atomique du rôle d'admin en fin de cycle
│   └── settings.php                  # Paramètres du compte administrateur
│
├── member/                           # ESPACE MEMBRE COTISANT (11 pages)
│   ├── dashboard.php                 # Tableau de bord personnel du membre (ou de la secrétaire)
│   ├── tontine.php                   # Règlement et liste des membres (sans numéro de CNI)
│   ├── contributions.php             # Mes cotisations + Paiement MoMo / Orange Money / Espèces
│   ├── payments.php                  # Historique de mes paiements (Part Bouffeur / Part Caisse)
│   ├── draw-rank.php                 # Mon rang de bouffeur et suivi de réception de mon pot
│   ├── calendar.php                  # Calendrier des échéances et dates de bouffe
│   ├── penalties.php                 # Mes pénalités et bonus
│   ├── meetings.php                  # Réunions publiées et accès direct au lien Zoom
│   ├── card.php                      # Ma carte de membre numérique avec QR Code sécurisé
│   ├── notifications.php             # Alertes et rappels calculés dynamiquement
│   └── profile.php                   # Mon profil, mon numéro MoMo/OM et mon mot de passe
│
└── (Racine : 8 fichiers publics & authentification)
    ├── index.php                     # Page d'accueil avec vidéo Hero et présentation
    ├── register-tontine.php          # Créer une tontine + Compte Admin (lance les 2 mois gratuits)
    ├── join-tontine.php              # Rejoindre une tontine via le code d'invitation
    ├── login.php                     # Connexion sécurisée + anti-force brute
    ├── logout.php                    # Déconnexion sécurisée
    ├── forgot-password.php           # Récupération de mot de passe
    ├── pricing.php                   # Détail du tarif SaaS (2 000 FCFA/mois par tontine)
    └── about-terms.php               # Fonctionnement détaillé et Conditions d'utilisation
```

3. Règles Métier & Circuit Financier à Respecter Impérativement

1. **Unicité des comptes et des tontines :**
   - Un utilisateur ne possède qu'un seul compte et ne peut appartenir qu'à **une seule tontine** (`users.tontine_id`).

- Un administrateur ne peut créer et diriger qu'**une seule tontine** (`tontines.admin_id UNIQUE`)

- * Flux d'adhésion sécurisé par approbation (`statut_validation`) :** Lorsqu'un membre s'inscrit via `join-tontine.php` avec le `code_invitation` de la tontine, son compte est créé avec `statut_validation = 'en_attente'` et il est redirigé vers `includes/waiting.php`. Tant que la secrétaire n'a pas approuvé sa demande (`statut_validation = 'accepte'`) dans `admin/members.php`, toute tentative de connexion sur `login.php` le renvoie vers `includes/waiting.php`..

2. **Double casquette de la Secrétaire (Admin & Cotisante) :**
   - La secrétaire gère la tontine et la caisse de réserve, mais elle est aussi **membre cotisante** : elle participe au tirage au sort, cotise à chaque séance et « bouffe » à son tour.
   - Elle bascule entre la gestion et son espace cotisant grâce aux boutons présents dans `includes/navbar.php` (`Mon Espace Cotisant ↔ Gestion Admin`).
3. **Le membre paie depuis son espace & Ventilation automatique de l'argent :**
   - **Cas n°1 (Cotisation normale du tour actif) :** 100 % du montant payé (`part_beneficiaire`) va directement au compte du **bouffeur du tour** (`part_caisse_reserve = 0`).
   - **Cas n°2 (Remboursement d'un retard d'une séance passée) :** Le paiement est divisé en deux parts :
     - `part_beneficiaire` : la cotisation principale impayée est versée à **l'ancien bouffeur de la séance concernée** (`beneficiaire_cycle_member_id`) qui n'avait pas reçu la totalité de son pot lors de son tour.
     - `part_caisse_reserve` : la pénalité de retard (ou le surplus) est versée dans la **caisse de réserve** gérée par la secrétaire (`tontines.solde_caisse_reserve`).
4. **Abonnement SaaS communautaire :**
   - À la création de la tontine, celle-ci bénéficie de **2 mois d'essai gratuits**.
   - À partir du 3e mois, l'administrateur paie **2 000 FCFA / mois** pour toute la tontine, ce qui donne un accès 100 % gratuit à tous les membres de sa tontine.

4. Guide du Design System (`style.css`) et Modèle de Page PHP

Pour que toutes les pages créées par les 5 membres passent automatiquement du **Mode Clair** au **Mode Sombre** sans aucun bug visuel, **n'utilisez jamais de couleurs en dur (`style="background: black"` ou `bg-white` sur les cartes)**. Utilisez exclusivement les classes de `assets/css/style.css` :

A. Les classes CSS obligatoires

Élément à afficher	Classe CSS à utiliser	Comportement Clair / Sombre
Bandeau de titre en haut d'une page	`<div class="page-header-dark">`	Dégradé Azur-Émeraude en Mode Clair / Bleu Nuit en Mode Sombre.
Carte standard (formulaire, stat, liste)	`<div class="card card-djangui hover-lift p-4">`	Blanche en Mode Clair / Gris Ardoise (`#111827`) en Mode Sombre + élévation au survol.
Carte vedette (Caisse, Pot, Abonnement)	`<div class="card-accent-djangui hover-lift p-4">`	Dégradé pastel en Mode Clair / Dégradé Bleu Nuit en Mode Sombre.
Sous-encadré à l'intérieur d'une carte	`<div class="p-3 rounded-3 box-adaptive">`	Gris très doux en Mode Clair / Ardoise en Mode Sombre.
Bordure colorée à gauche d'une carte	`.border-left-navy`, `.border-left-emerald`, `.border-left-warning`, `.border-left-danger`	Ajoute une barre verticale de 5px à gauche de la carte.
Bouton d'action principal (Bleu)	`<button class="btn btn-glow-blue">`	Dégradé Bleu Roi avec ombre lumineuse.
Bouton de validation / paiement (Vert)	`<button class="btn btn-glow-emerald">`	Dégradé Vert Émeraude avec ombre lumineuse.
Tableau de données	`<table class="table table-djangui align-middle">`	En-tête stylisé et lignes adaptatives au thème.

B. Squelette type d'une page PHP (À copier pour créer chaque nouvelle page)

Toutes les pages des dossiers `admin/` et `member/` doivent respecter cette structure en **3 étapes** :

```
<?php
// Exemple : admin/cycles.php (ou member/contributions.php)
require_once __DIR__ . '/../includes/db.php';
require_once __DIR___. '/../includes/functions.php';

// 1. SÉCURITÉ : verifier_admin() pour le dossier /admin/ | verifier_connexion() pour /member/
verifier_admin();

$tontine_id = $_SESSION['tontine_id'];
```

```
    $user_id    = $_SESSION['user_id'];

    // 2. TRAITEMENT DU FORMULAIRE (SI POST)
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verifier_csrf();
    // Requêtes PDO préparées ici ($pdo->prepare(...))
    message_flash('success', 'Opération enregistrée avec succès.');
    header('Location: ' . BASE_URL . '/admin/cycles.php');
    exit;
}

// 3. RÉCUPÉRATION DES DONNÉES (SELECT VIA PDO)
$stmt = $pdo->prepare("SELECT * FROM cycles WHERE tontine_id = ? ORDER BY numero_cycle DESC");
$stmt->execute([$tontine_id]);
$cycles = $stmt->fetchAll();

// 4. AFFICHAGE HTML (HEADER + NAVBAR + CONTENU + FOOTER)
$titre_page = "Gestion des Cycles";
require_once __DIR__ . '/../includes/header.php';
require_once __DIR__. '/../includes/navbar.php';
?>

<!-- Bandeau de titre adaptatif -->
<div class="page-header-dark">
    <div class="container">
        <h2 class="fw-bold mb-1">Gestion des Cycles</h2>
        <p class="text-muted mb-0 small">Configurez et suivez les cycles successifs de votre tontine</p>
    </div>
</div>

<div class="container pb-5">
    <div class="card card-djangui p-4">
        <?= champ_csrf() ?>
        <!-- Contenu Bootstrap 5 de la page -->
    </div>
</div>

<?php require_once __DIR__. '/../includes/footer.php'; ?>
```

5. Répartition du Travail entre les 5 Membres (Zéro conflit GitHub)

Voici la **nouvelle répartition équilibrée entre les 5 membres du groupe** (4 à 5 fichiers par personne, regroupés par logique métier pour que chacun comprenne facilement sa partie et travaille sur sa branche GitHub sans aucun conflit) :

1. Tableau récapitulatif de la nouvelle répartition (23 fichiers pour 5 membres)

Membre	Module fonctionnel attribué	Fichiers à coder (4 à 5 fichiers chacun)	Tables SQL utilisées	Branche GitHub
Membre 1	**Pilotage Admin, Membres & Infos Tontine**	1. admin/dashboard.php 2. admin/members.php 3. admin/tontine-info.php 4. admin/settings.php 5. member/tontine.php	tontines, users, cycles, payments	lot1-admin-membres

Membre 2	**Cycles, Tirage au Sort, Rang & Bilans**	1. admin/cycles.php 2. admin/draws.php 3. admin/reports.php 4. member/draw-rank.php 5. member/calendar.php	cycles, cycle_members, contributions	lot2-cycles-tirage
Membre 3	**Cotisations, Paiements Ventilés & Accueil Membre**	1. member/dashboard.php 2. member/contributions.php 3. member/payments.php 4. admin/contributions.php	contributions, payments, cycle_members, tontines	lot3-cotisations-paiements

Membre	Module fonctionnel attribué	Fichiers à coder (4 à 5 fichiers chacun)	Tables SQL utilisées	Branche GitHub
Membre 4	**Caisse de Réserve, Pénalités, Abonnement SaaS & Passation**	1. admin/penalties.php 2. member/penalties.php 3. admin/subscription.php 4. admin/transfer.php	penalties, tontines, subscriptions, admin_transfers	lot4-caisse-saas-transfert
Membre 5	**Réunions Zoom, Carte QR Code, Alertes & Profil**	1. admin/meetings.php 2. member/meetings.php 3. member/card.php 4. member/notifications.php 5. member/profile.php	meetings, users, contributions, penalties	lot5-reunions-qrcode-profil

2. Détail précis du travail de chacun des 5 membres

MEMBRE 1 : Pilotage Admin, Gestion des Membres & Infos Tontine (5 fichiers)

- admin/dashboard.php` :Affiche les indicateurs clés du cycle actif (montant attendu, montant collecté, taux de cotisation en %, nombre de membres à jour et en retard, solde de la caisse de réserve, nom du bouffeur du tour et les 5 derniers paiements).
- admin/members.php` :** Permet à la secrétaire de contrôler les adhésions et de gérer les membres de sa tontine :

  Affichage, copie rapide (pour WhatsApp/SMS) et possibilité de régénérer le code_invitation` de la tontine.

  Validation des demandes d'adhésion en attente (`statut_validation = 'en_attente'`) :** la secrétaire vérifie le nom, le téléphone et le `numero_cni` du demandeur, puis clique sur **Accepter** (`statut_validation = 'accepte'`) ou **Refuser** (`statut_validation = 'refuse'`).

  Consultation de la liste des membres validés avec affichage protégé de leur `numero_cni` (visible uniquement sur cette page Admin) et modification de leur statut (`actif`, `inactif`, `sorti`).

  (Note sur le portail public géré par le Membre 1 : sur `join-tontine.php`, le nouveau membre remplit toutes ses informations, y compris son `numero_cni` et son mot de passe, puis atterrit sur `includes/waiting.php` jusqu'à validation par l'admin).

- admin/tontine-info.php` :Permet de modifier le nom, la ville et le règlement de la tontine, d'afficher ou renouveler le `code_invitation` à partager, et d'arrêter définitivement la tontine (`statut = 'fermee'`) en fin de cycle.
- admin/settings.php` :Permet à la secrétaire de modifier ses coordonnées personnelles, son numéro MoMo/OM de gestion de la caisse et son mot de passe.
- member/tontine.php` :Vue côté membre des informations de la tontine, du solde de la caisse de réserve et de la liste des membres actifs (**sans jamais afficher le numéro de CNI**).

(Note sur le portail public géré par le Membre 1 : sur `join-tontine.php`, le nouveau membre saisit lui-même son `numero_cni` et son propre mot de passe en utilisant le `code_invitation` reçu ; il n'y a donc plus de changement de mot de passe forcé à la 1re connexion dans `login.php`).

MEMBRE 2 : Cycles, Tirage au Sort, Calendrier & Rapports (5 fichiers)

- admin/cycles.php : Permet de créer un nouveau cycle (brouillon), de cocher les membres qui participent à ce cycle (y compris la secrétaire) dans cycle_members, d'activer le cycle (si tous les paramètres sont complets et qu'aucun autre cycle n'est actif) et de clôturer un cycle terminé sans effacer l'historique.

- `admin/draws.php` : Exécute le tirage au sort aléatoire unique pour le cycle actif, attribue les rangs `1..N` sans doublon dans `cycle_members.rang_beneficiaire`, génère automatiquement toutes les séances de cotisation dans `contributions` (en associant chaque séance à son bouffeur `beneficiaire_cycle_member_id`) et verrouille le tirage (`tirage_verrouille = 1`).
- `admin/reports.php` : Affiche le bilan financier complet de chaque cycle (actifs et clôturés) avec bouton d'impression.

- member/draw-rank.php : Affiche au membre son numéro de tour (rang_beneficiaire), la date prévue de sa « bouffe », la barre de progression de son pot (montant_pot_recu / montant_pot_attendu) et l'ordre général du cycle.
- member/calendar.php : Présente le calendrier chronologique des séances du cycle avec la date limite de paiement et le bénéficiaire de chaque tour.

MEMBRE 3 : Cotisations, Paiements Ventilés & Tableau de Bord Membre (4 fichiers)

- member/dashboard.php : Tableau de bord personnel du membre connecté (et de la secrétaire lorsqu'elle clique sur « *Mon Espace Cotisant* ») : affiche sa prochaine cotisation à payer, son rang au tirage, l'état de son pot et la prochaine réunion Zoom.
- member/contributions.php : Liste les cotisations du membre séance par séance et permet de payer par MTN MoMo, Orange Money ou Espèces. Applique la règle de ventilation automatique :

- Si c'est la cotisation normale du tour : 100 % va dans `part_beneficiaire` pour le bouffeur du tour.
- Si c'est le remboursement en retard d'une séance passée avec pénalité : la dette de cotisation (`part_beneficiaire`) est versée à **l'ancien bouffeur de cette séance** (et met à jour son `montant_pot_recu`), tandis que la pénalité ou le surplus (`part_caisse_reserve`) augmente le `solde_caisse_reserve` de la tontine.

- `member/payments.php` : Affiche tous les reçus de paiement du membre avec le détail exact (`montant_total`, `part_beneficiaire` versée au bouffeur et `part_caisse_reserve` versée à la caisse).

- `admin/contributions.php` : Permet à la secrétaire de filtrer toutes les cotisations par séance ou par statut (`paye`, `partiel`, `en_retard`, `en_attente`) et de voir combien il reste à percevoir pour chaque ancien bouffeur.

MEMBRE 4 : Caisse de Réserve, Pénalités/Bonus, Abonnement SaaS & Passation (4 fichiers)

- `admin/penalties.php` : Affiche le solde de la **Caisse de réserve**, permet à la secrétaire d'imputer une pénalité ou d'enregistrer un bonus avec motif obligatoire, et de marquer une pénalité comme réglée (`regle`, ce qui ajoute le montant à `solde_caisse_reserve`) ou annulée (`annule`).
- `member/penalties.php` : Permet au membre de consulter les pénalités et bonus qui le concernent (avec motif, date et auteur) et de régler une pénalité en attente directement vers la caisse de réserve.
- `admin/subscription.php` : Affiche l'état des **2 mois d'essai gratuit** rattachés à la tontine, la date d'expiration, l'historique des paiements SaaS et permet de renouveler l'abonnement mensuel à **2 000 FCFA / mois** pour toute la tontine (en mode démonstration identifié par `statut_paiement = 'simule'`).
- `admin/transfer.php` : Permet en fin de cycle de transférer de manière atomique (`$pdo->beginTransaction()`) le rôle d'administrateur à un autre membre actif de la tontine, tout en conservant l'historique dans `admin_transfers`.

MEMBRE 5 : Réunions Zoom, Carte QR Code, Notifications & Profil (5 fichiers)

- admin/meetings.php` :Permet à la secrétaire de programmer une réunion (titre, date/heure, lien Zoom, ordre du jour) et de changer son statut (`brouillon`, `publiee`, `terminee`, `annulee`).
- member/meetings.php` :Affiche aux membres toutes les réunions publiées avec le bouton direct pour rejoindre la réunion sur Zoom.
- member/card.php` :Affiche la carte de membre numérique officielle de la tontine avec un **QR Code généré automatiquement** contenant uniquement l'identifiant public (`code_public`) et le statut du membre, sans jamais exposer le mot de passe ni la CNI.
- member/notifications.php` :Génère dynamiquement les alertes du membre connecté (cotisation proche de l'échéance ou en retard, pénalité en attente, résultat du tirage au sort disponible, nouvelle réunion Zoom publiée).
- member/profile.php` :Permet au membre de consulter ses informations personnelles et de modifier à tout moment son canal de paiement préféré (`momo`, `om`, `especes`), son numéro Mobile Money / Orange Money de réception du pot ainsi que son mot de passe.

6. Protocole de Travail en Équipe sur GitHub (Pas à Pas)

Étape 1 : Initialisation du dépôt (Fait une seule fois par le Membre 1)

Le **Membre 1** place le socle commun (`database/`, `assets/`, `includes/`, `index.php`, `ARCHITECTURE_ET_EQUIPE.md`) dans `htdocs/djangui/` et le pousse sur la branche `main` :

```
git init
git add .
git commit -m "Socle initial Djangui : BDD, Design Bi-Theme, Navbar et Index"
git branch -M main
git remote add origin https://github.com/VOTRE_COMPTE/djangui.git
git push -u origin main
```

Étape 2 : Récupération et création de sa branche (Chaque membre 1 à 5)

Chacun des 5 collègues clone le projet dans son dossier `C:\xampp\htdocs\djangui\`, importe `database/djangui_complet.sql` dans son phpMyAdmin local, puis crée **sa propre branche** :

```
# Cloner le projet dans htdocs
cd C:/xampp/htdocs
git clone https://github.com/VOTRE_COMPTE/djangui.git
cd djangui

# Créer et basculer sur sa branche personnelle (ex: pour le Membre 2)
git checkout -b feature-membre2-admin-gestion
```

Étape 3 : Routine quotidienne de travail d'un membre

Avant de commencer à coder et à la fin de chaque journée :

```
# 1. Récupérer les dernières mises à jour validées de la branche main
git pull origin main

# 2. Après avoir codé et testé ses pages sur http://localhost/djangui/
git add .
git commit -m "Ajout de admin/dashboard.php et admin/members.php"
git push origin feature-membre2-admin-gestion
```

Étape 4 : Les 4 Règles d'Or pour éviter tout conflit Git

1. **Ne travaillez jamais directement sur `main`** : chacun code sur sa branche `feature-membreX-...` et ouvre une *Pull Request* sur GitHub pour fusionner dans `main`.
2. **Ne modifiez pas les fichiers d'un autre lot** : si le Membre 3 ou 4 a besoin d'une petite classe CSS supplémentaire ou d'une fonction dans `includes/functions.php`, il le demande au **Membre 1** qui l'ajoute proprement pour tout le monde.
3. **Testez systématiquement les 4 vues avant de faire un `git push` :**
   - Vue **PC en Mode Clair** ( ) ;
   - Vue **PC en Mode Sombre** ( ) ;
   - Vue **Smartphone (`F12` < 992px) en Mode Clair et Sombre** avec la barre du bas `.mobile-bottom-nav`.
4. **Sécurité obligatoire sur chaque fichier :**
   - Toujours utiliser `$pdo->prepare(...)` avec `?` (jamais de variable PHP concaténée dans du SQL).
   - Toujours entourer les variables affichées en HTML par `<?= e($variable) ?>`.
   - Toujours mettre `<?= champ_csrf() ?>` dans chaque `<form method="POST">` et `verifier_csrf()` en haut du traitement POST.
